

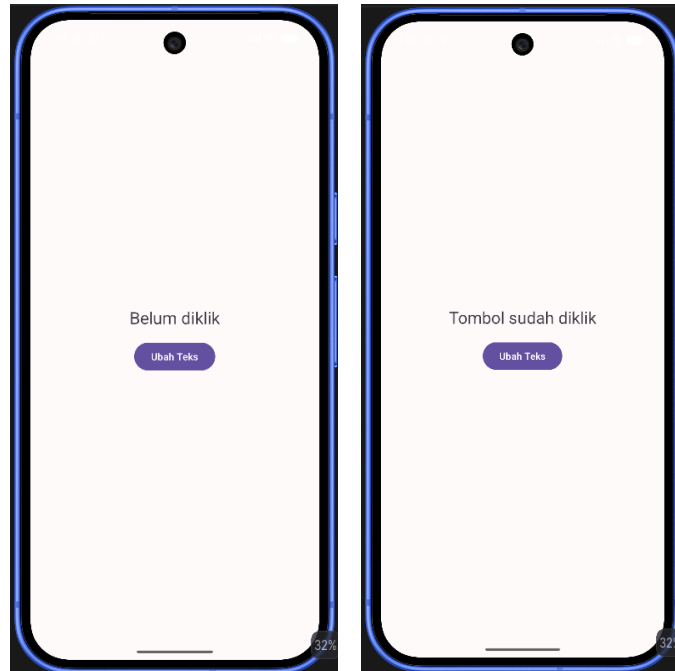
Nama : Janice Kusuma
NIM : 245150400111021
Kelas : PAM-SI-D

JAWABAN MODUL 3

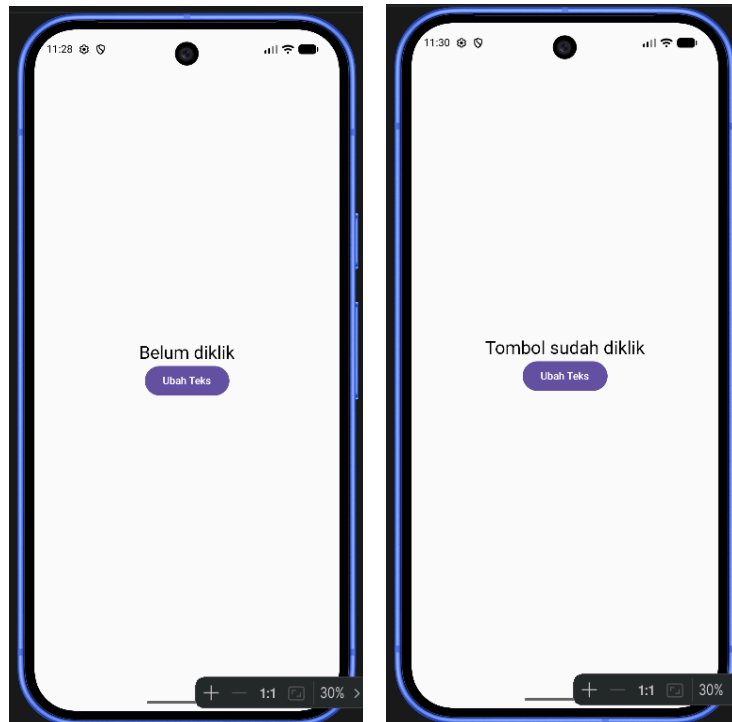
Perbandingan Imperatif dan Deklaratif dalam Perubahan Teks

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik.

Project ImperativeXmlApp



Project DeclarativeComposeApp



2. **Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya.**

Ketika tombol diklik, objek yang diubah adalah TextView yang ada pada variabel tvMessage. Baris kodenya adalah

```
val tvMessage = findViewById<TextView>(R.id.tvMessage)
btnChange.setOnClickListener {
    tvMessage.text = "Tombol sudah diklik"
}
```

3. **Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya.**

Pada project compose, nilai yang diubah ketika tombol diklik adalah objek message atau state dari message.

```
Button(onClick = {
    message = "Tombol sudah diklik"
}) {
    Text("Ubah Teks")
}
```

4. **Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()?**

Karena dalam Jetpack Compose, program mengubah state dan mendeklarasikan tampilan berdasarkan state, bukan mengubah properti komponen UI secara langsung.

5. **Jelaskan alur perubahan pada XML/View dengan pola: event -> View -> perubahan tampilan.**

Alurnya dimulai saat pengguna menekan tombol, kemudian onClick listener menerima event klik yang dilakukan oleh user, lalu program mendapatkan referensi TextView (tvMessage), dimana properti tvMessage tersebut diubah dan alhasil teks pada layar juga berubah.

6. Jelaskan alur perubahan pada Compose dengan pola: event -> state -> recomposition -> perubahan tampilan.

Dimulai ketika pengguna menekan tombol, kemudian onClick dijalankan yang membuat nilai state message berubah, karena adanya perubahan state, maka terjadi rekomposisi UI yang menggunakan message, sehingga Text ditampilkan dengan nilai baru.

7. Apa yang terjadi jika message diganti menjadi variabel biasa: var message = "Belum diklik"? Jelaskan penyebabnya.

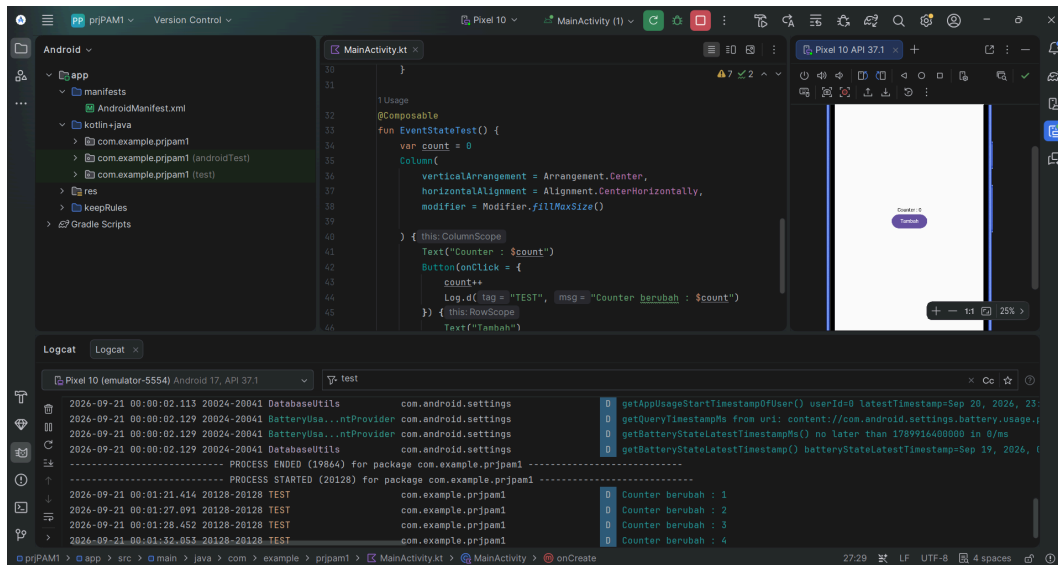
Jika message menjadi variabel biasa yaitu belum diklik dan kemudian ditekan oleh pengguna, nilai message akan berubah, namun tidak otomatis mengganti tampilan menjadi tombol sudah diklik. Hal ini karena variabel biasa bukan merupakan **observable state** yang dapat dipantau oleh Compose.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

Dari yang saya pahami, pendekatan imperatif dan deklaratif memiliki cara memperbarui antarmuka yang cukup berbeda. Pada pendekatan imperatif seperti XML/view, harus mencari atau mendapatkan referensi komponen yang akan diubah lalu mengubah secara langsung komponen UI-nya, sedangkan pendekatan deklaratif, programmer tidak perlu mengubah komponen UI secara langsung, tapi mengubah state yang menjadi sumber data tampilan. etika state berubah, Compose akan melakukan recomposition dan memperbarui bagian UI yang membutuhkan perubahan secara otomatis.

Event Handling State

- 1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.**



2. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?

Setiap kali tombol “Tambah” diklik, akan ada pesan “Counter berubah : 1”, “Counter berubah : 2”, dan seterusnya. Angka counter akan terus bertambah setiap diklik.

3. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?

Tidak terjadi apa-apa dan tidak ada yang berubah

4. Apa yang menyebabkan hal tersebut terjadi?

Tidak ada yang berubah pada tampilan aplikasi karena tidak ada bagian kode yang akan menyebabkan perubahan UI, hanya ada deklarasi `count = 0` saja, tanpa mendeteksi perubahan state dan melakukan sesuatu terhadap perubahan state tersebut.

5. Langkah selanjutnya adalah mengubah kode sebelumnya untuk menggunakan state yaitu pada Composable EventStateTest.

6. Ubah kode program pada bagian fungsi Composable EventStateTest dari yang sebelumnya:

```
var count = 0
```

menjadi:

```
var count by remember { mutableStateOf(0) }
```

```
@Composable
fun EventStateTest() {
    var count by remember { mutableStateOf( value = 0) }
    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d( tag = "TEST", msg = "Counter berubah : $count")
        }) { this.RowScope
            Text("Tambah")
        }
    }
}
```

7. Berikut kode lengkap EventStateTest:

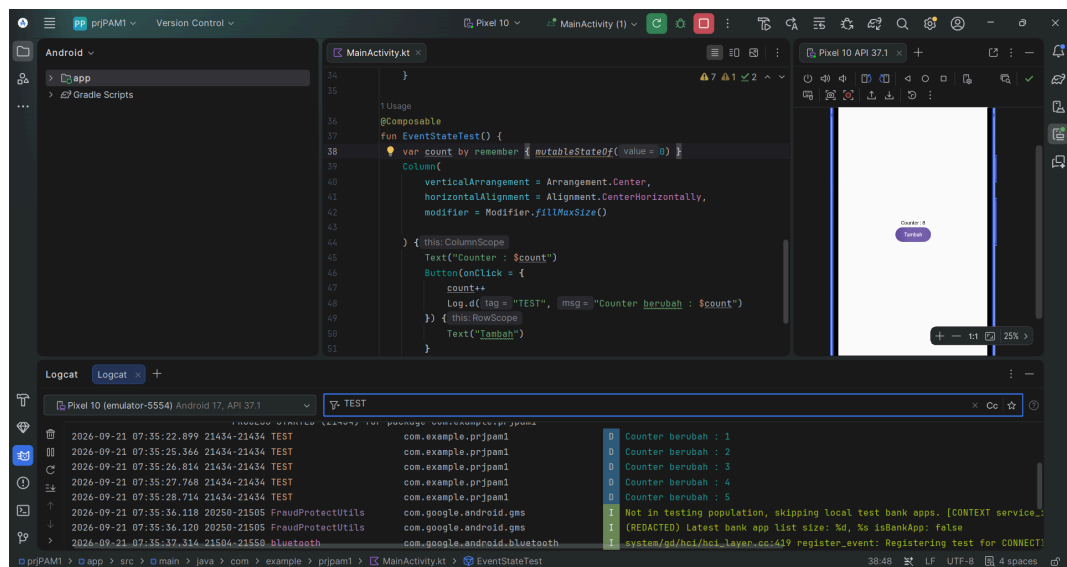
```
@Composable
fun EventStateTest() {
```

```

var count by remember { mutableStateOf(0) }
Column(
    verticalArrangement = Arrangement.Center,
    horizontalAlignment = Alignment.CenterHorizontally,
    modifier = Modifier.fillMaxSize()
) {
    Text("Counter : $count")
    Button(onClick = {
        count++
        Log.d("TEST", "Counter berubah : $count")
    }) {
        Text("Tambah")
    }
}
}

```

8. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



9. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?
Setiap kali tombol "Tambah" di-klik, akan muncul hal yang sama seperti sebelumnya, yaitu terdapat pesan log berupa "Counter berubah : 1", "Counter berubah : 2", "Counter berubah : 3", dst sebanyak berapa kali tombol "Tambah" diklik
10. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
Sekarang, tampilan pada aplikasi adalah counter nya ikut berubah dan bertambah setiap kali tombol "Tambah" di klik.

11. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?

Hal ini disebabkan oleh perbedaan cara deklarasi count, dimana kode diubah menjadi `var count by remember { mutableStateOf(0) }`, sehingga count menjadi state yang diamati dalam compose; sehingga ketika tombol diklik, compose melakukan recomposition dan menampilkan UI yang baru, bersama deklarasi `mutableStateOf()` juga. Remember juga digunakan untuk “meingingat” angka sebelumnya sehingga tidak di-reset setiap recomposition.

12. Apa yang dimaksud recomposition pada Compose?

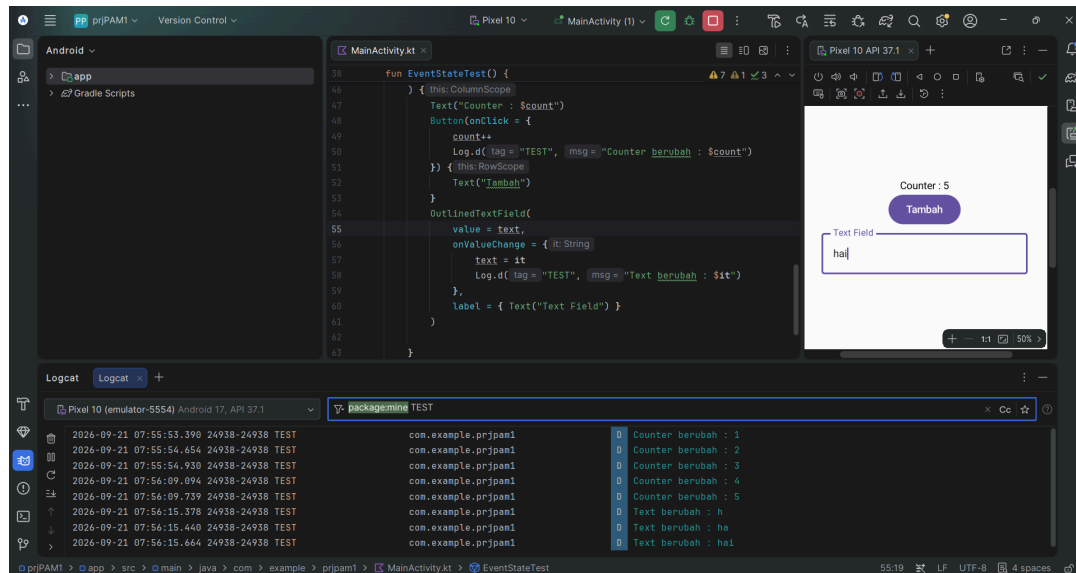
Recomposition adalah proses ketika Jetpack Compose menjalankan kembali bagian composable yang bergantung pada state yang berubah untuk memperbarui UI. Recomposition dilakukan ketika ada state yang berubah dalam composable.

13. Langkah selanjutnya adalah menambah kode sebelumnya dengan menambah state baru yaitu text bertipe string pada Composable EventStateTest. Kemudian di dalam Column panggil composable OutlinedTextField di bawah Button Tambah. Penambahan kode program membuat kode pada fungsi Composable EventStateTest menjadi:

```
fun EventStateTest() {
    var count by remember { mutableStateOf(0) }
    var text by remember { mutableStateOf("") }

    Column(
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally,
        modifier = Modifier.fillMaxSize()
    ) {
        Text("Counter : $count")
        Button(onClick = {
            count++
            Log.d("TEST", "Counter berubah : $count")
        }) {
            Text("Tambah")
        }
        OutlinedTextField(
            value = text,
            onValueChange = {
                text = it
                Log.d("TEST", "Text berubah : $it")
            },
            label = { Text("Text Field") }
        )
    }
}
```

14. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



15. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?
Log akan mencatat secara urut/nyambung setiap karakter yang dimasukkan. Misalnya saya memasukkan kata “hai”, pada logcat mencatat “Text berubah: h”, “Text berubah: ha”, “Text berubah: hai”.
16. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field?
Tampilan pada text field sesuai dengan inputan huruf yang diberikan. Tampilan akan menyimpan input terbaru dan digunakan kembali sebagai isi TextField.
17. Apa yang menyebabkan hal tersebut terjadi?
Hal ini bisa terjadi karena text dideklarasikan sebagai state Compose. Ketika text field diisi, terdapat perubahan nilai yang akan men-trigger event onChange, kemudian mengubah nilai state text. Karena state berubah, Compose melakukan recomposition, kemudian OutlinedTextField membuat nilai value = text terbaru, alhasil karakter yang diketik dapat langsung ditampilkan pada UI.
18. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?
Terdapat 2 state, yaitu `var count by remember { mutableStateOf(0) }` dan `var text by remember { mutableStateOf("") }`, dimana count merupakan state bertipe integer untuk menyimpan nilai counter, dan text adalah state berupa string untuk menyimpan isi OutlinedTextField.

19. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?

Terdapat 2 event, yaitu onClick pada button “Tambah” yang menangani event ketika tombol diklik, dan onValueChange pada OutlinedTextField yang menangani event ketika text field diisi atau terdapat perubahan input.

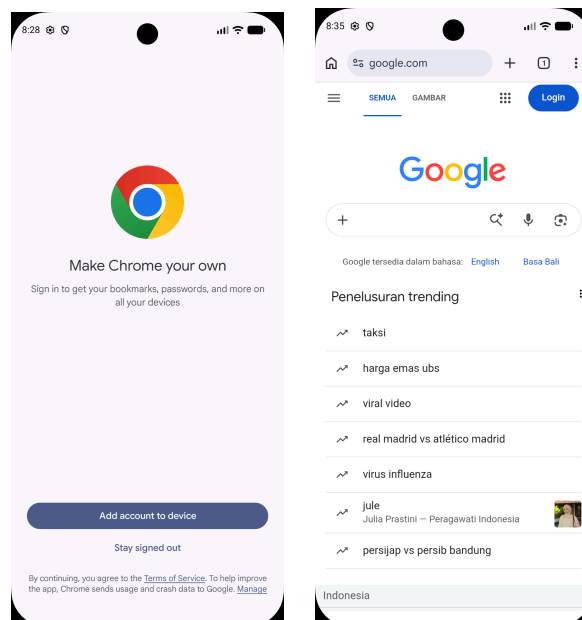
20. Kapan event - event tersebut dijalankan?

Event onClick pada button akan dijalankan ketika tombol “Tambah” di klik, dan event onValueChange akan dijalankan ketika Text Field diisi text atau text field berubah nilainya.

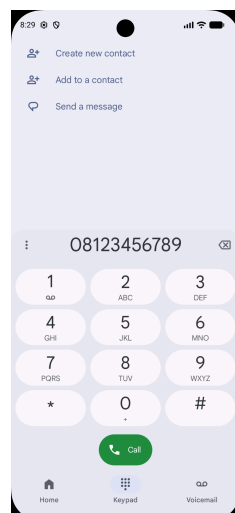
Implisit Intent

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Lakukan pengujian pada setiap tombol:

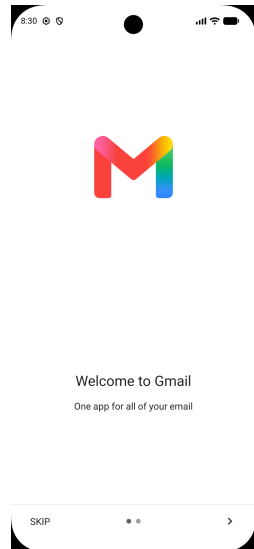
a. Klik Buka Browser



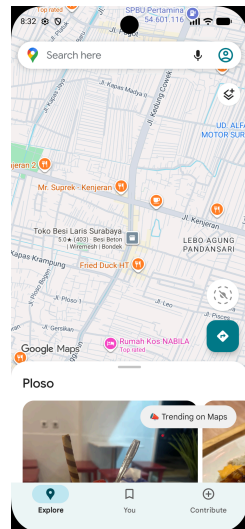
b. Klik Buka Telepon



c. Klik Kirim Email



d. Klik Buka Maps



2. Amati apa yang terjadi pada setiap aksi, kemudian ambil screenshot hasilnya
3. Pada kode yang digunakan, terdapat beberapa action,
4. Apa perbedaan fungsi dari masing-masing action intent tersebut?

Perbedaan ketiga action tersebut terletak pada jenis aktivitas atau aksi yang ingin dilakukan, untuk tujuan spesifiknya ditentukan bersama dengan data pada intent.

a. `Intent.ACTION_VIEW`

Action intent ini berfungsi untuk menampilkan atau membuka suatu data dengan aplikasi yang sesuai. Contohnya pada percobaan, intent ini digunakan untuk membuka link google, sehingga Android mencari aplikasi yang dapat membuka URL web tersebut.

b. `Intent.ACTION_DIAL`

Action intent dial ini digunakan untuk membuka aplikasi *dialer* telepon dengan nomor tertentu. Pada percobaan, dibuat dial tersebut dengan

`Uri.parse("tel:08123456789")`, yang kemudian membuat Android membuka aplikasi telepon dengan nomor tersebut dalam kondisi sudah dimasukkan.

c. Intent.ACTION_SENDTO

Intent ACTION_SENDTO ini berfungsi untuk membuka aplikasi yang dapat mengirim data ke tujuan tertentu, seperti contohnya pada percobaan, ke gmail.

5. Pada kode yang digunakan, terdapat beberapa data seperti:

```
data = Uri.parse("mailto:test@gmail.com")
data = Uri.parse("https://www.google.com")
data = Uri.parse("geo:-7.250445,112.768845")
```

6. Apa fungsi data pada intent?

Data seperti contoh di atas, berfungsi untuk memberikan informasi atau objek yang harus diproses oleh action dari Intent. Pada contoh di atas, berarti data yang ingin dibuka atau digunakan adalah email: test@gmail.com, membuka [google.com](https://www.google.com), dan menampilkan data geografis tersebut.

7. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

Pada aplikasi, action dan data bekerja sama untuk menentukan aplikasi yang cocok untuk menampilkan intent. Action menentukan apa yang ingin dilakukan, sedangkan data menentukan terhadap apa aksi tersebut dilakukan.